Pipeline RAG Multi-Dominio con Scraping Concurrente y Aislamiento Vectorial

Sistemas Paralelos y Distribuidos - Informe de Rendimiento

Repositorio: https://github.com/luzclarita03/lexbancario-ai

1. Arquitectura y concurrencia

```
POST /api/ingestar-web
 FASE 1  Descarga HTTP ......... asyncio.gather + Semaphore(p) / httpx (I/O-bound)
 FASE 2  Limpieza + Chunking ... ThreadPoolExecutor(p) via asyncio.to_thread (CPU)
 FASE 3  Embeddings por lotes .. gemini-embedding-001 (batch=10) + backoff 429
 FASE 4  Insercion masiva ...... Supabase / pgvector (upsert por lote)
                              |
                              v
         public.normativa_bancaria  (aislamiento logico por coleccion_id)

POST /api/consultar {pregunta, coleccion_id, top_k}
 embed(pregunta) -> RPC match_normativa_coleccion(..., p_coleccion_id) -> contexto filtrado
              -> Gemini (temperature=0.0) -> respuesta + fuentes
```

La descarga usa asyncio + httpx (I/O-bound); la limpieza/chunking (CPU) se despacha a un ThreadPoolExecutor para no bloquear el event loop. La escritura en Supabase es centralizada tras el gather, por lo que no hay condiciones de carrera; las URLs caidas se reportan en urls_fallidas.

2. Aislamiento vectorial (SQL / RPC)

```
alter table public.normativa_bancaria add column if not exists coleccion_id varchar(50);
create index if not exists normativa_bancaria_coleccion_id_idx
    on public.normativa_bancaria using btree (coleccion_id);

create or replace function public.match_normativa_coleccion(
    query_embedding vector(768), p_coleccion_id varchar(50),
    match_threshold float default 0.35, match_count int default 4)
returns table (..., coleccion_id varchar(50), similarity float)
language plpgsql stable as $$
begin
  return query
  select ..., 1 - (n.embedding <=> query_embedding) as similarity
  from public.normativa_bancaria n
  where n.embedding is not null
    and n.coleccion_id = p_coleccion_id             -- aislamiento determinista
    and 1 - (n.embedding <=> query_embedding) > match_threshold
  order by n.embedding <=> query_embedding
  limit match_count;
end; $$;
```

El parametro se llama p_coleccion_id para no colisionar con la columna homonima de RETURNS TABLE (evita el error PostgreSQL 42725). Prueba de cero contaminacion cruzada (scripts/test_aislamiento.py): PASS -> cada coleccion solo devuelve sus filas; una coleccion inexistente devuelve 0; y match_normativa (sin filtro) mezcla ambas.

3. Metodologia de medicion

Corpus: 10 URLs fijas de tamano similar (7.9k-24.9k caracteres). Fases instrumentadas: (1) scraping/red, (2) chunking/CPU, (3) API embeddings, (4) insercion Supabase. Parametros: chunk_size=1000, chunk_overlap=200, workers p en {1,2,4,8}. T_1 = 1 worker; S_p = T_1/T_p; E_p = S_p/p. Chunks por corrida ~ 211. Se presentan dos campanas: (a) embeddings API real (p=1,2; cuota gratuita agotada); (b) barrido completo con --embed-mode mock para caracterizar el escalado.

3.1 Campana - API real de embeddings

p	T_p (s)	%red	%chunk	%emb	Speedup	Eficiencia
1	156.01	~1.3%	0.2%	~98.5%	1.000	100.0%
2	122.57	~1.3%	0.2%	~98.5%	1.273	63.6%

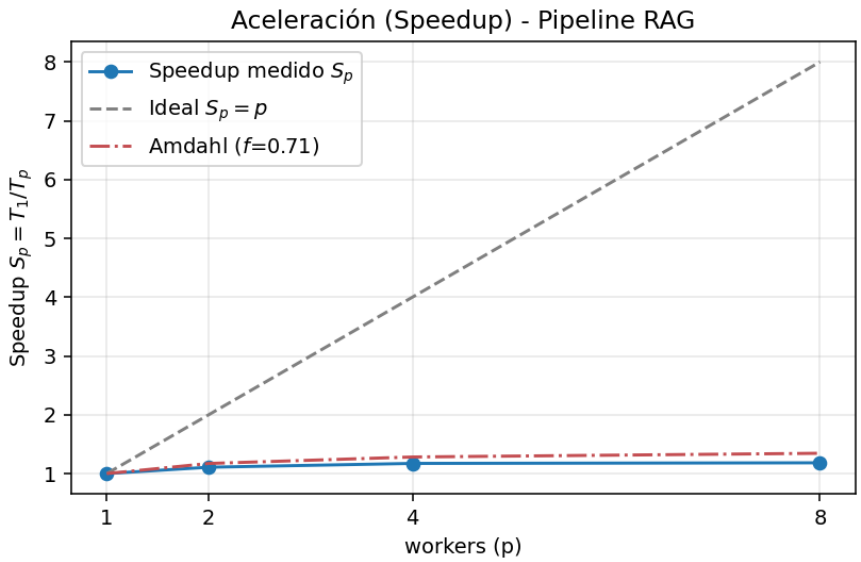
3.2 Campana - barrido completo (embeddings mock)

p	T_p (s)	S_p	E_p (%)	%red	%chunk	%emb	%DB
1	31.23	1.000	100.0	7.1	1.0	70.7	21.3
2	28.19	1.108	55.4	4.7	1.0	78.3	15.9
4	26.63	1.173	29.3	3.6	1.5	82.9	12.0
8	26.39	1.183	14.8	4.3	1.9	83.6	10.2

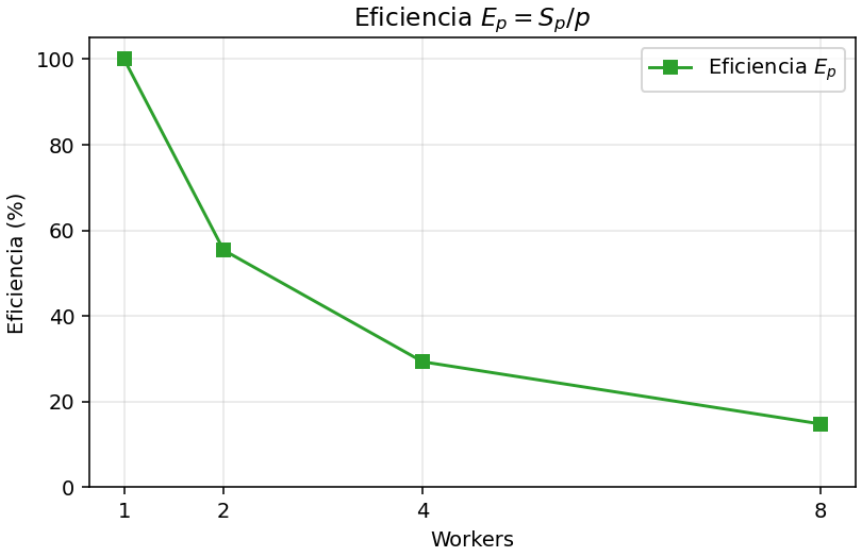
Los embeddings dominan ~70-98% del tiempo total. La insercion mejora por batching; el scraping satura ~p=4 y se degrada en p=8 por contencion de red.

4. Speedup y Eficiencia

Speedup S_p = T_1/T_p

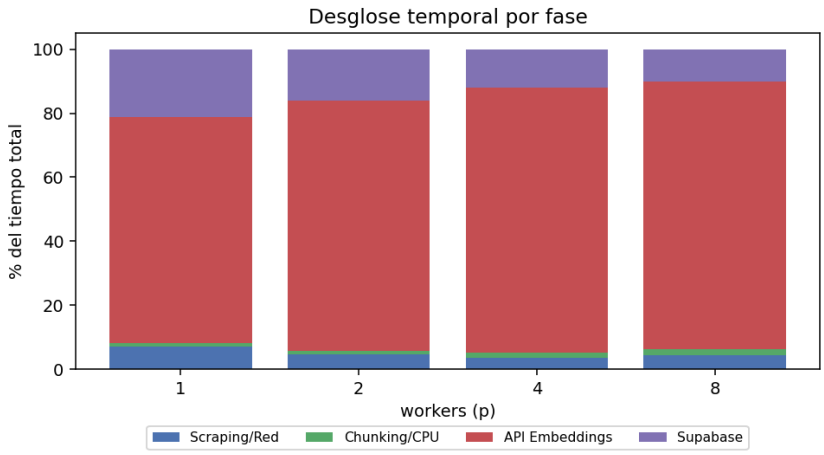


Eficiencia E_p = S_p/p

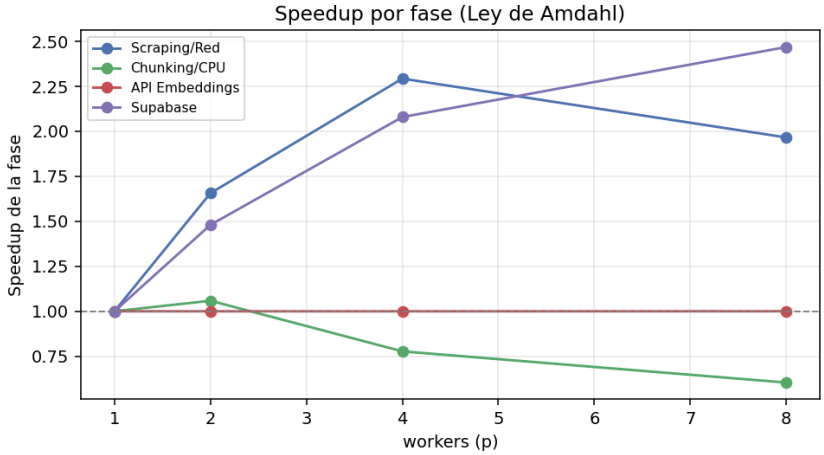


5. Desglose del paralelismo y Ley de Amdahl

Distribucion porcentual del tiempo por fase



Speedup por fase (amortizacion del paralelismo)



Fraccion no paralelizable f ~ 0.71-0.84 (embeddings de API externa + insercion). Con f=0.80 el limite teorico S_max = 1/0.8 = 1.25 coincide con el speedup medido (1.18-1.27): el techo lo impone la API externa de embeddings, no la CPU. La fase de red acelera hasta p=4 (S~2.3) y sufre en p=8 por contencion.

6. Conclusiones

1. Aislamiento vectorial determinista: la columna+indice B-Tree + RPC PL/pgSQL restringe la busqueda coseno a la coleccion, con prueba de cero contaminacion cruzada (PASS).

2. Concurrencia efectiva: asyncio + semaforo + thread pool + batching aceleran el scraping (S~2.3 con p=4) y hacen el pipeline viable para lotes de URLs.

3. Techo de escalabilidad: la API externa de embeddings limita S_p cerca de 1/f (Ley de Amdahl), confirmado por los datos.

Recomendaciones: cache de embeddings (implementada en --embed-mode cache), modelo local de embeddings y control de tasa adaptativo.

6.1 Reproduccion

```
docker compose up --build
python scripts/test_aislamiento.py
python benchmark.py --embed-mode api   --workers 1,2,4,8 --reps 3
python benchmark.py --embed-mode cache --workers 1,2,4,8
python benchmark.py --embed-mode mock  --workers 1,2,4,8
python scripts/graficar.py && python scripts/generar_informe.py
```

Repositorio del proyecto: https://github.com/luzclarita03/lexbancario-ai